

ATIVIDADE AVALIATIVA AV2 MÁQUINAS UNIVERSAIS, TURING E λ -CÁLCULO TEORIA DA COMPUTABILIDADE

Prof. Daniel Leal Souza – Semestre: 01/2026

Disciplina	Teoria da Computabilidade
Turmas	CC5MA e CC5NA
Tema geral	Pesquisa, formalização, implementação, documentação em GitHub e defesa oral de modelos relacionados a máquinas universais, modificações da Máquina de Turing, Máquina de Post, λ -Cálculo, Funções Recursivas de Kleene e Autômatos de Pilha.
Equipes	Até 4 integrantes : escolher exatamente 2 máquinas/modelos distintos . Equipes com 5 integrantes : escolher exatamente 3 máquinas/modelos .
Entrega	CC5MA: 10/06/2026 CC5NA: 12/06/2026 . até 23h59.
Entregáveis	Link obrigatório do repositório GitHub + slides + arquivos JFLAP .jff e/ou código-fonte + README + relatório curto de rastreamento de execução + referências + declaração de uso de IA.
Pontuação	Até 2,0 pts na AV2. A nota poderá ser individualizada em caso de ausência, baixa participação ou incapacidade de explicar o trabalho.
Apresentação	Seminário presencial, com presença obrigatória. Tempo de apresentação: de 12 a 15 minutos por equipe , incluindo demonstração e arguição.

1. Resumo objetivo das instruções do trabalho

Esta AV2 será um **seminário técnico com implementação**. Cada equipe de até **4 integrantes** deverá selecionar **duas máquinas/modelos distintos** da Seção 3, estudar seus fundamentos, apresentar suas definições formais, explicar sua relação com computabilidade e entregar **uma implementação bem elaborada para cada máquina/modelo selecionado**.

Equipes com **5 integrantes**, quando autorizadas pelo professor, deverão selecionar **três máquinas/modelos distintos** e entregar **3 implementações no total**. A ampliação é requisito para que a equipe maior possa alcançar a nota máxima (mais gente, mais trabalho).

As implementações poderão ser feitas no **JFLAP** ou em **linguagem de programação**. Cada implementação deverá resolver um **problema diferente**. Não é permitido que uma equipe entregue duas máquinas/modelos resolvendo o mesmo problema com pequenas mudanças de notação, alfabeto ou entrada. Equipes diferentes podem escolher as mesmas máquinas/modelos, mas seus problemas, exemplos, arquivos, códigos, transições e demonstrações devem ser próprios e diferentes.

Todo trabalho deverá estar obrigatoriamente em um **repositório GitHub da equipe**. O link do repositório deverá ser entregue no Google Classroom e deverá permanecer acessível ao professor até o encerramento da correção. O GitHub deve conter os arquivos implementados, slides ou link dos slides, README, testes, rastreamentos, referências e declaração de uso de IA.

O item mais importante da avaliação será o **domínio conceitual e técnico**. A equipe deverá explicar o formalismo, justificar as decisões de modelagem, demonstrar que os exemplos pertencem ao modelo escolhido e responder a perguntas durante a arguição. Trabalhos visualmente bons, mas conceitualmente frágeis, copiados, triviais, gerados por IA sem revisão ou não compreendidos pelos integrantes, receberão descontos severos.

2. Finalidade e objetivos

A atividade busca consolidar os conceitos de **máquinas universais**, **Máquina de Turing**, **modificações da Máquina de Turing**, **Máquina de Post**, **não determinismo**, **autômatos linearmente limitados**, **Funções Recursivas de Kleene**, **Autômatos de Pilha** e **λ -Cálculo**. Ao final, a equipe deverá ser capaz de:

- explicar a origem, motivação e definição formal de cada máquina/modelo escolhido;
- relacionar cada máquina/modelo à Hipótese de Church-Turing, à noção de algoritmo e aos limites da computabilidade;
- construir implementações próprias, suficientemente complexas e reproduzíveis;
- demonstrar o funcionamento por meio de entradas, passos relevantes, saídas e rastreamento de execução;
- organizar o projeto no GitHub, com arquivos, instruções, testes e evidências suficientes para reprodução e auditoria;
- responder a perguntas conceituais e práticas sobre qualquer parte do trabalho.

3. Máquinas/modelos disponíveis para escolha

Cada equipe deverá pesquisar e escolher máquinas/modelos da lista abaixo, composta por **nove opções**. A escolha deve ser indicada nos slides, no README e no Classroom. A repetição de máquina/modelo entre equipes é permitida, mas **não** autoriza repetição de exemplos, problemas, códigos, arquivos `.jff`, tabelas de transição ou explicações.

Opção	Máquina/modelo e foco esperado
1	Máquina de Turing para Autômato Linearmente Limitado (ALL) . Fita limitada pela entrada, relação com linguagens sensíveis ao contexto, critérios de aceitação e comparação com a Máquina de Turing padrão.
2	Máquina de Turing com Building Blocks . Construção modular por blocos/submáquinas, composição de comportamentos, reuso e encadeamento de processamento.
3	Máquina de Turing com Múltiplas Fitas . Formalização do modelo multifita, organização da computação, exemplos de processamento e discussão da simulação por fita única.
4	Máquina de Turing Não Determinística . Múltiplas transições possíveis, árvore de computação, aceitação por existência de ramo aceitante e comparação com máquinas determinísticas.
5	Outras modificações sobre a Máquina de Turing . Fita bilateral, múltiplas cabeças, fita multidimensional, movimento estacionário, modelos teóricos (e.g. quântico), experimentais ou outra extensão formalmente justificada.
6	Máquina de Post . Modelo baseado em fila, regras de manipulação, reconhecimento/processamento, relação com computabilidade e comparação com a Máquina de Turing.
7	λ-Cálculo . Sintaxe, abstração, aplicação, redução beta, funções puras, variáveis livres/ligadas e implementação de avaliador, simulador ou exemplos de redução em linguagem de programação.
8	Funções Recursivas de Kleene . Funções iniciais, composição, recursão primitiva, minimização, distinção entre função parcial e total, exemplos de construção funcional e relação com computabilidade.
9	Autômatos de Pilha . Modelo com controle finito e pilha, operações de empilhar/desempilhar, reconhecimento de linguagens livres de contexto, comparação com autômatos finitos e limites em relação à Máquina de Turing.

4. Quantidade obrigatória de máquinas/modelos e implementações

A quantidade exigida é fixa. Não há substituição por slides adicionais, pesquisa textual maior ou apresentação mais longa.

Tamanho da equipe	Escolha obrigatória	Implementação obrigatória
Até 4 integrantes	Exatamente 2 máquinas/modelos distintos da Seção 3.	Exatamente 1 implementação bem elaborada (complexidade média ou alta) para cada máquina/modelo; total de 2 implementações .
5 integrantes	Exatamente 3 máquinas/modelos distintos da Seção 3.	Exatamente 1 implementação bem elaborada (complexidade média ou alta) para cada máquina/modelo; total de 3 implementações .

Para todos os casos, as implementações da mesma equipe devem resolver **problemas diferentes**. Exemplo de regra: uma equipe não pode implementar duas variações de reconhecedor para a mesma linguagem com troca apenas de símbolos; também não pode resolver o mesmo problema com dois modelos diferentes sem justificar uma comparação formal substancial, previamente aceita pelo professor.

5. Desenvolvimento obrigatório por máquina/modelo escolhido

Para cada máquina/modelo selecionado, a equipe deverá entregar e apresentar:

1. **Pesquisa conceitual:** contextualização histórica e técnica, com referências confiáveis.
2. **Formalização:** estados, alfabetos, fita(s), fila, função/relação de transição, regras de redução, critério de parada/aceitação ou componente equivalente.
3. **Problema próprio:** descrição clara do problema resolvido, da linguagem reconhecida, da função computada, da transformação realizada ou da propriedade demonstrada.
4. **Implementação:** arquivo `.jff` ou código-fonte executável, organizado e compatível com o formalismo estudado.
5. **GitHub:** disponibilização do projeto em repositório GitHub, com organização suficiente para localizar, executar e auditar cada implementação.
6. **Rastreamento de execução:** ao menos **três entradas de teste**, incluindo casos aceitos/produzidos corretamente e casos rejeitados, inválidos ou de fronteira, quando aplicável.
7. **Análise técnica:** explicação do que é computado, como a computação evolui, por que o exemplo não é trivial e quais limitações foram observadas.

6. Implementação: JFLAP ou linguagem de programação

As equipes poderão usar **JFLAP**, **linguagem de programação** ou uma combinação dos dois. A escolha da ferramenta não reduz o nível de exigência.

- **JFLAP:** entregar arquivos `.jff`, imagens ou tabelas de transição nos slides, demonstração de execução e os respectivos arquivos no GitHub.
- **Programação:** entregar código-fonte organizado no GitHub, README, instruções de execução, dependências, exemplos de entrada e saída, comentários mínimos e separação clara entre o modelo formal e a rotina de simulação/interpretação.
- **Todos os nove modelos podem ser implementados por programação.** Exemplos: simulador de ALL, simulador de MT com múltiplas fitas, gerador de árvore para MT não determinística, simulador de Máquina de Post, avaliador de expressões lambda ou sistema que execute building blocks.

- A implementação programada deve representar explicitamente o formalismo estudado. Não será aceito usar apenas uma função pronta da linguagem de programação para produzir a resposta final sem simular ou representar o modelo.

7. Requisitos mínimos de complexidade e originalidade

- Em máquinas baseadas em estados, cada implementação deverá ter **mais de 8 estados** e complexidade compatível com AV2. Recomenda-se no mínimo **10 transições relevantes** por máquina.
- Para λ -Cálculo, cada implementação deverá conter ao menos **7 etapas relevantes de redução, avaliação ou interpretação**, ou a equipe deverá implementar um avaliador/interpretador funcional de expressões lambda com exemplos documentados.
- Não é permitido reutilizar exemplos resolvidos em sala de aula, exemplos dos slides, exercícios demonstrados pelo professor ou adaptações de baixo impacto desses exemplos.
- Não serão aceitos exemplos como: reconhecedor de uma única cadeia fixa, troca isolada de um símbolo, incremento artificial com poucos passos, cópia de máquina pronta sem alteração estrutural ou duas variações superficiais do mesmo exemplo.
- Cada implementação deve resolver um problema diferente das demais implementações da equipe. A diferença deve estar no problema computacional e na construção formal, não apenas no nome dos estados, nos símbolos ou na estética dos slides.

8. Uso de inteligência artificial e rastreabilidade de autoria

O uso de ferramentas de IA é permitido apenas como apoio. A IA não pode substituir estudo, implementação, análise ou domínio conceitual. A equipe deverá entregar um arquivo `uso_ia.pdf` ou seção específica no README contendo:

- ferramenta utilizada e data aproximada de uso;
- finalidade: estudo, revisão textual, depuração, geração inicial de ideias, organização de slides, etc.;
- resumo dos prompts utilizados e dos trechos aproveitados;
- explicação do que foi modificado, corrigido ou rejeitado pela equipe;
- declaração de que todos os integrantes revisaram e compreendem os trechos incorporados.

Durante a arguição, o professor poderá solicitar que qualquer integrante explique decisões de modelagem, transições, trechos de código, resultados de testes ou conceitos formais. Incapacidade de explicar o próprio material será considerada evidência de baixa autoria ou uso inadequado de IA.

9. GitHub e estrutura mínima do repositório

Todo trabalho deverá possuir um **repositório GitHub obrigatório**. A entrega no Classroom deverá conter o link do repositório. O repositório poderá ser público ou privado, desde que o professor tenha acesso integral até o encerramento da correção. Links inacessíveis, repositórios vazios ou repositórios sem os arquivos essenciais serão tratados como entrega incompleta.

O repositório deve conter, no mínimo:

- `README.md` com turma, integrantes, máquinas/modelos escolhidos, problemas resolvidos, instruções de execução, dependências e exemplos de uso;
- pasta ou seção para `slides/`, contendo os slides ou link claramente identificado para a apresentação;
- pasta ou seção para `implementacoes/`, contendo os arquivos `.jff` e/ou códigos-fonte;
- pasta ou seção para `testes/` ou `rastreamento/`, contendo entradas, saídas esperadas, saídas obtidas e passos relevantes de execução;
- arquivo `uso_ia.pdf`, `uso_ia.md` ou seção equivalente no README, mesmo quando a equipe declarar que não utilizou IA;

Alterações feitas no repositório após o prazo de entrega poderão ser desconsideradas. Caso a equipe corrija algo após o prazo, deverá deixar isso explícito no README ou em registro de versão, sem apagar a versão entregue dentro do prazo.

10. Dinâmica de apresentação para todas as equipes

- A apresentação será presencial e obrigatória nas datas indicadas: **10/06/2026 para CC5MA e 12/06/2026 para CC5NA.**
- Tempo de apresentação: de **12 até 15 minutos. Recomendação:** 3 minutos de teoria, 6 minutos de demonstração das implementações, 3 minutos de análise e até 3 minutos de arguição inicial.
- Todos os integrantes devem falar ou demonstrar parte identificável do trabalho.
- A equipe deve estar preparada para executar os exemplos ao vivo ou apresentar evidências reprodutíveis caso haja falha técnica justificada.

11. Entregáveis no Google Classroom

A submissão deverá conter:

- link obrigatório do repositório GitHub da equipe, acessível ao professor;
- slides em PDF, PPTX, Google Slides ou equivalente, no GitHub e/ou no Classroom;
- arquivos .jff e/ou código-fonte no repositório GitHub;
- README com instruções de execução, dependências, exemplos de entrada e saída;
- relatório curto de rastreamento de execução, com tabelas de teste para cada implementação;
- referências consultadas;
- arquivo ou seção de declaração de uso de IA;
- identificação completa de turma, máquinas/modelos escolhidos e integrantes.

12. Rubrica de avaliação - até 2,0 pts

Critério	Valor	Descrição
1. Domínio conceitual e arguição	0,45	Clareza, precisão técnica, capacidade de explicar cada formalismo escolhido, responder perguntas, justificar escolhas e demonstrar compreensão individual.
2. Formalização dos modelos	0,30	Definição correta dos componentes formais de cada máquina/modelo, critérios de aceitação/parada, relação com Máquina de Turing, computabilidade e limitações.
3. Implementações e funcionamento	0,35	Implementações executáveis, reproduzíveis, coerentes com os modelos escolhidos e demonstradas com entradas, passos relevantes e saídas.
4. Complexidade, originalidade e testes	0,30	Problemas diferentes, exemplos não triviais, próprios, com mais de 8 estados quando aplicável, rastreamento de execução e testes adequados.
5. Seminário, organização e participação	0,20	Slides objetivos, tempo adequado, divisão equilibrada, postura acadêmica e participação identificável de todos os integrantes.
6. GitHub, entrega, documentação e rastreabilidade	0,40	Repositório GitHub obrigatório e acessível, README completo, arquivos organizados, referências, registro de IA, rastreamentos, estrutura reprodutível e entrega correta no Classroom.
Total	2,00	Pontuação máxima da atividade avaliativa.

13. Penalidades e critérios de anulação (gradual ou total)

As penalidades poderão ser acumuladas, conforme a gravidade.

Ocorrência	Consequência possível
Trabalhos, máquinas, códigos, slides, exemplos ou problemas substancialmente similares entre equipes, sem justificativa técnica independente.	Desconto de até 50% da nota ou anulação das partes similares.
Cópia de arquivos .jff, código, slides, textos ou exemplos prontos sem citação e sem contribuição própria.	Desconto severo, anulação da implementação ou nota zero no trabalho, conforme gravidade.
Plágio textual, plágio de código, plágio de máquina, compra de trabalho ou autoria falsa.	Nota zero e encaminhamento conforme normas acadêmicas institucionais, quando aplicável.
Uso de IA sem declaração, uso de IA para gerar solução não compreendida ou apresentação de conteúdo que a equipe não consegue explicar.	Desconto em domínio, documentação e/ou implementação; em casos graves, anulação da parte correspondente.
Exemplos triviais, já vistos em aula, copiados dos slides ou com adaptações apenas cosméticas.	Perda dos pontos de complexidade/originalidade e possível invalidação do exemplo.
Entrega de duas ou mais implementações que resolvem o mesmo problema ou problemas equivalentes com mudanças superficiais.	Invalidação de uma das implementações e perda proporcional dos pontos de implementação, testes e originalidade.
Equipe de 5 integrantes que entregar apenas duas máquinas/modelos ou duas implementações.	Nota máxima limitada a 1,6 pts , antes de outros descontos.
Ausência de apresentação presencial ou ausência sem justificativa de integrante.	A entrega não substitui o seminário; nota individual poderá ser reduzida.
Arquivos não executáveis, links inacessíveis, ausência de README ou impossibilidade de reproduzir as implementações.	Perda dos pontos de implementação e documentação, conforme extensão do problema.
Ausência de repositório GitHub, link do GitHub não informado no Classroom ou repositório inacessível ao professor.	Perda do critério de GitHub/documentação e nota máxima limitada a 1,6 pts , antes de outros descontos.
Repositório GitHub vazio, sem README, sem arquivos essenciais, sem testes/rastreamentos ou sem correspondência com o que foi apresentado.	Desconto proporcional no critério de GitHub/documentação e, se inviabilizar a reprodução, perda também em implementação.
Alterações feitas no GitHub após o prazo sem identificação clara no README ou sem preservação da versão entregue no prazo.	As alterações posteriores poderão ser desconsideradas e a equipe poderá perder pontos de rastreabilidade.

15. Referências sugeridas

- DIVERIO, Tiarajú Asmuz; MENEZES, Paulo Blauth. **Teoria da Computação: Máquinas Universais e Computabilidade**. 3. ed. Porto Alegre: Bookman, 2011.
- MENEZES, Paulo Blauth. **Linguagens Formais e Autômatos**. 6. ed. Porto Alegre: Bookman, 2011.
- Slides da disciplina sobre Máquina de Turing, máquinas universais, modificações da Máquina de Turing, Máquina de Post, solucionabilidade e λ -Cálculo.
- Documentação do JFLAP, quando utilizado.

16. Checklist final

- ☐ O tema, a turma e todos os integrantes estão identificados.
- ☐ O link do repositório GitHub foi entregue no Classroom e está acessível ao professor.
- ☐ O GitHub contém README, slides ou link dos slides, implementações, testes, rastreamentos, referências e declaração de uso de IA.
- ☐ A equipe de até 4 integrantes escolheu exatamente duas máquinas/modelos distintos.
- ☐ A equipe de 5 integrantes escolheu exatamente três máquinas/modelos distintos.
- ☐ Há uma implementação própria, bem elaborado (complexidade média ou alta) e reproduzível para cada máquina/modelo escolhido.
- ☐ Cada implementação resolve um problema diferente das demais implementações da equipe.
- ☐ Cada máquina possui mais de 8 estados, quando aplicável.
- ☐ Há testes, rastreamento de execução e interpretação dos resultados para cada implementação.
- ☐ Os arquivos `.jff` e/ou códigos foram entregues corretamente no GitHub.
- ☐ O README permite executar os exemplos.
- ☐ As referências foram citadas.
- ☐ O uso de IA foi declarado, ou foi informado que não houve uso.
- ☐ Todos os integrantes conseguem explicar teoria, implementação e resultados.